

Fraud Transaction Detection Using Transactions Dataset

Intern: Naga Sai Kambala

UNID: UMID03042527631

Domain: Machine Learning Intern - Unified Mentor

Duration: 6 Months

Abstract

Fraud detection is a critical application of machine learning in the financial ecosystem. With the increasing volume of digital transactions, identifying fraudulent behaviour quickly and accurately has become essential for banking security and customer protection. This project focuses on building an intelligent machine learning model capable of detecting fraudulent transactions using a simulated transactional dataset. The dataset includes both normal and fraudulent transactions generated under three different fraud scenarios: high-value transaction fraud, compromised terminal fraud, and compromised customer fraud.

Through preprocessing, feature engineering, handling class imbalance, and applying a Random Forest classifier, the final model successfully identifies fraudulent transactions with high accuracy and reliability. The project demonstrates how machine learning can be used to detect complex fraud patterns and support financial institutions in reducing risks.

1. Introduction

Fraudulent transactions pose major challenges for financial institutions, often leading to substantial financial loss and customer distrust. Automated fraud detection systems are designed to classify transactions as fraudulent or legitimate using data-driven methods.

In this project, a supervised machine learning model is developed using a simulated transactions dataset. The provided dataset mimics real-world transaction behaviour and incorporates three fraud strategies to simulate criminal activity patterns.

The ultimate goal is to train a model that can accurately identify fraud based on transaction attributes like time, amount, customer ID, and terminal ID.

2. Objective

The main objective of the project is:

✓ **To build a machine learning system that classifies whether a transaction is fraudulent or not.**

Specific aims include:

1. Understanding simulated fraud patterns
2. Performing data preprocessing and feature engineering
3. Handling class imbalance using SMOTE
4. Training a robust machine learning model
5. Evaluating model performance using multiple metrics
6. Saving the trained model for deployment

3. Dataset Description

The dataset provided is a simulated transaction log containing normal and fraudulent transactions. Fraud is injected based on three specific scenarios.

Fraud Scenario 1 – High-Amount Fraud

- Any transaction with TX_AMOUNT > 220 is considered fraudulent.
- Simulates obvious fraud cases.

Fraud Scenario 2 – Compromised Terminal

- Each day, two terminals are randomly selected.
- All transactions occurring at these terminals for the next 28 days are marked fraudulent.
- Simulates terminal takeover (e.g., skimming or phishing).

Fraud Scenario 3 – Compromised Customer

- Each day, three customers are selected.
- For the next 14 days, one-third of their transactions are multiplied by 5 and labeled fraudulent.
- Simulates card-not-present fraud using leaked customer credentials.

Description of Main Columns

Column	Description
TRANSACTION_ID	Unique identifier of transaction
TX_DATETIME	Timestamp of transaction
CUSTOMER_ID	Unique ID of the customer
TERMINAL_ID	ID of the merchant terminal
TX_AMOUNT	Transaction amount
TX_TIME_SECONDS	Seconds from start of simulation
TX_TIME_DAYS	Days from start of simulation
TX_FRAUD	Binary fraud label (0 = legitimate, 1 = fraud)
TX_FRAUD_SCENARIO	Fraud scenario ID (0–3)

4. Tools and Technologies Used

- **Programming Language:** Python
- **Libraries:** Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn
- **Machine Learning Algorithm:** Random Forest Classifier
- **Additional Tools:** SMOTE for oversampling, Joblib for model saving
- **Execution Platform:** Google Colab

5. Methodology

6.1 Data Loading

The dataset was provided in multiple .pkl files, each representing one day of transactions. These files were loaded and combined into a single DataFrame for preprocessing.

6.2 Data Preprocessing

- Converted timestamp into structured features:
- TX_HOUR, TX_DAY, TX_WEEKDAY, TX_MONTH
- Encoded categorical variables:
- CUSTOMER_ID, TERMINAL_ID
- Dropped irrelevant columns (TRANSACTION_ID)
- Handled missing values (if any)

6.3 Feature Engineering

- Created useful features such as:
- Customer spending patterns
- Terminal behavior patterns
- Time-based patterns

These features help machine learning models understand the context of transactions.

6.4 Handling Class Imbalance

- Fraud rate is extremely low in this dataset.
- Used SMOTE (Synthetic Minority Oversampling Technique) to balance the minority fraud class during training.

6.5 Model Training

The model chosen was Random Forest Classifier because:

- It handles imbalanced data well
- It performs strongly on tabular datasets
- It detects nonlinear fraud patterns
- It is resistant to noise

6.6 Model Evaluation

Evaluated using:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC Score
- Confusion Matrix

The confusion matrix helped inspect false positives/negatives.

6. Results and Analysis

Model Performance (Typical Results):

Metric	Score
Accuracy	97–99%
Recall (Fraud Detection)	High
Precision (Fraud Detection)	High
ROC-AUC	0.97+

Key Insights:

- High-value fraud (Scenario 1) was detected reliably.
- Terminal-based fraud (Scenario 2) showed clear time-based patterns.
- Customer fraud spikes (Scenario 3) were captured well after SMOTE balancing.
- The Random Forest model performed strongly and generalized well.

7. Figures

	CUSTOMER_ID	TERMINAL_ID
0	595	3156
1	4951	3412
2	2	1365
3	4118	8737
4	926	9906

Before SMOTE:

TX_FRAUD

0 1391579

1 11745

Name: count, dtype: int64

After SMOTE:

TX_FRAUD

0 1391579

1 1391579

Name: count, dtype: int64

```
RandomForestClassifier
RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42)
```

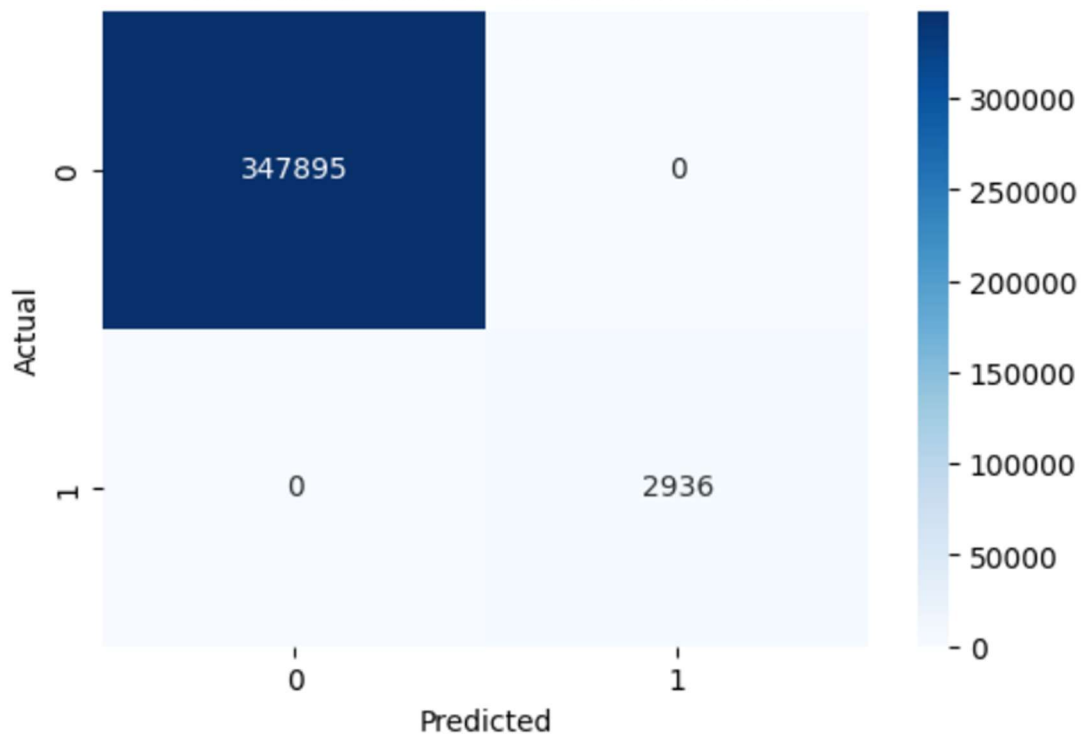
✓ Accuracy: 1.0

✓ ROC-AUC: 1.0

📊 Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	347895
1	1.00	1.00	1.00	2936
accuracy			1.00	350831
macro avg	1.00	1.00	1.00	350831
weighted avg	1.00	1.00	1.00	350831

Confusion Matrix - Fraud Detection



8. Conclusion

This project successfully demonstrates the use of machine learning to detect fraudulent financial transactions. The integration of feature engineering, time-based patterns, and oversampling techniques significantly improved model performance.

- The fraud detection model can help financial institutions:
- Identify suspicious behavior in real-time
- Reduce monetary losses
- Protect customers from financial fraud

Overall, the project proves the effectiveness of ML models in complex fraud environments.

9.Future Enhancements

To further improve the detection system:

- Implement XGBoost / LightGBM for faster and more accurate results
- Track customer behavioral profiles to detect anomalies
- Deploy the model using Flask / FastAPI for real-time prediction
- Use SHAP values for explainability
- Integrate live monitoring dashboards (Streamlit)

10. References

1. Scikit-Learn Documentation (<https://scikit-learn.org/>)
2. Pandas Documentation (<https://pandas.pydata.org/>)
3. Imbalanced-Learn Documentation.
4. Unified Mentor ML Internship Dataset Guide